

GUÍA DE LABORATORIO N.º 02

Configuración de endpoints y controladores. Dependency Injection en Spring Boot

Curso: Desarrollo Web Integrado (100000ST61)

Semana	2
Unidad de aprendizaje	Unidad 1: API REST
Duración sugerida	3 horas académicas
Tecnologías	Java 25, Maven, Spring Boot, Visual Studio Code, Postman o Thunder Client
Producto de la práctica	API REST de productos con endpoints GET, POST, PUT y DELETE, organizada con Controller, Service y DI

1. Logro de aprendizaje de la práctica

Al finalizar la práctica, el estudiante configura endpoints REST en Spring Boot mediante controladores, aplica inyección de dependencias por constructor y separa responsabilidades entre controller, service y modelo para construir una API básica de productos.

2. Situación problemática

Una tienda necesita exponer una API REST inicial para gestionar productos. En esta primera versión todavía no se conectará a una base de datos; los datos se mantendrán en memoria para concentrar el aprendizaje en la comunicación HTTP, la construcción de endpoints, el diseño de controladores y el uso correcto de servicios con inyección de dependencias.

3. Conceptos teóricos necesarios

3.1. ¿Qué es un endpoint?

Un endpoint es una URL expuesta por el backend para que un cliente, como un navegador, Postman o una aplicación Angular, pueda solicitar o enviar información. En una API REST, cada endpoint representa una operación sobre un recurso.

Método HTTP	Endpoint	Uso	Ejemplo de resultado
GET	/api/productos	Listar productos	Devuelve arreglo JSON
GET	/api/productos/{id}	Buscar producto por id	Devuelve un objeto JSON
POST	/api/productos	Crear producto	Devuelve producto creado
PUT	/api/productos/{id}	Actualizar producto	Devuelve producto actualizado
DELETE	/api/productos/{id}	Eliminar producto	Devuelve 204 No Content

3.2. ¿Qué es un controlador REST?

Un controlador REST es una clase que recibe solicitudes HTTP y devuelve respuestas. En Spring Boot se declara con `@RestController`. Esta anotación combina el comportamiento de un controlador con la respuesta directa en el cuerpo HTTP, usualmente JSON.

```
@RestController
@RequestMapping("/api/productos")
public class ProductoController {
    // Métodos que atienden solicitudes HTTP
}
```

3.3. ¿Qué es Dependency Injection?

Dependency Injection, o inyección de dependencias, es una técnica donde una clase no crea manualmente los objetos que necesita. En lugar de usar `new ProductoService()`, Spring crea y administra el objeto `ProductoService` como bean, y lo entrega al controlador mediante el constructor.

```
private final ProductoService service;

public ProductoController(ProductoService service) {
    this.service = service;
}
```

Buena práctica

En Spring Boot moderno se recomienda la inyección por constructor porque hace explícitas las dependencias, permite usar `final`, facilita pruebas unitarias y evita dependencias ocultas.

3.4. Responsabilidades por capa

Capa	Responsabilidad	Ejemplo en esta práctica
Controller	Recibe solicitudes HTTP y devuelve	ProductoController

	respuestas	
Service	Contiene la lógica de negocio	ProductoService
Model / DTO	Representa datos de entrada y salida	Producto, ProductoRequest

4. Requisitos previos

- Java 25 instalado y configurado en PATH.
- Visual Studio Code instalado.
- Extensiones: Extension Pack for Java y Spring Boot Extension Pack.
- Maven disponible desde terminal.
- Postman o Thunder Client instalado para probar POST, PUT y DELETE.
- Proyecto de Spring Boot creado en la semana 1 o crear uno nuevo para esta práctica.

5. Crear o verificar el proyecto Spring Boot

Si se continúa desde la guía de la semana 1, abrir ese proyecto en VS Code. Si se creará un proyecto nuevo, usar Spring Initializr con la siguiente configuración:

Campo	Valor recomendado
Project	Maven
Language	Java
Spring Boot	Versión estable disponible en Spring Initializr
Java	25
Packaging	Jar
Group	com.utp
Artifact	semana2
Dependencies	Spring Web, Spring Boot DevTools opcional

Importante

Para esta semana no se requiere MySQL ni Spring Data JPA. La base de datos se trabajará después, cuando el sílabo aborde backend con bases de datos.

6. Estructura esperada del proyecto

```
src/main/java/com/utp/semana2
├── Semana2Application.java
├── controller
│   └── ProductoController.java
├── dto
│   ├── ProductoRequest.java
│   └── ErrorResponse.java
├── exception
│   ├── ProductoNoEncontradoException.java
│   └── GlobalExceptionHandler.java
├── model
│   └── Producto.java
└── service
    └── ProductoService.java
```

7. Paso a paso de implementación

7.1. Configurar el puerto y nombre de aplicación

Abrir el archivo src/main/resources/application.properties y agregar:

```
spring.application.name=semana2
server.port=8080
```

7.2. Crear el modelo Producto

Crear el paquete model y dentro el archivo Producto.java. Esta clase representa el recurso principal de la API.

```
package com.utp.semana2.model;

public class Producto {

    private Long id;
    private String nombre;
    private double precio;
    private int stock;

    public Producto() {
    }

    public Producto(Long id, String nombre, double precio, int stock) {
        this.id = id;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public int getStock() {
        return stock;
    }

    public void setStock(int stock) {
        this.stock = stock;
    }
}
```

7.3. Crear el DTO ProductoRequest

Crear el paquete dto y dentro el archivo ProductoRequest.java. Un DTO permite separar los datos que recibe la API de la clase interna que usa el backend.

```
package com.utp.semana2.dto;

public record ProductoRequest(
    String nombre,
    double precio,
    int stock
) {
}
```

¿Por qué usar record?

Java permite usar records para representar datos inmutables de forma breve. En esta práctica se usa ProductoRequest como objeto de

entrada para POST y PUT.

7.4. Crear la excepción personalizada

```
package com.utp.semana2.exception;

public class ProductoNoEncontradoException extends RuntimeException {

    public ProductoNoEncontradoException(Long id) {
        super("No existe un producto con id: " + id);
    }
}
```

7.5. Crear el servicio ProductoService

Crear el paquete service y dentro el archivo ProductoService.java. Este servicio concentra la lógica de negocio. Por ahora usará una estructura en memoria, suficiente para practicar endpoints y dependency injection.

```
package com.utp.semana2.service;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

import org.springframework.stereotype.Service;

import com.utp.semana2.dto.ProductoRequest;
import com.utp.semana2.exception.ProductoNoEncontradoException;
import com.utp.semana2.model.Producto;

@Service
public class ProductoService {

    private final Map<Long, Producto> productos = new ConcurrentHashMap<>();
    private final AtomicLong secuencia = new AtomicLong(0);

    public ProductoService() {
        crear(new ProductoRequest("Laptop Lenovo", 3500.00, 8));
        crear(new ProductoRequest("Mouse Logitech", 80.00, 25));
        crear(new ProductoRequest("Teclado mecánico", 240.00, 12));
    }

    public List<Producto> listar() {
        return new ArrayList<>(productos.values());
    }

    public Producto buscarPorId(Long id) {
        Producto producto = productos.get(id);
        if (producto == null) {
            throw new ProductoNoEncontradoException(id);
        }
        return producto;
    }

    public List<Producto> buscarPorNombre(String nombre) {
        String texto = nombre.toLowerCase();
        return productos.values().stream()
            .filter(p -> p.getNombre().toLowerCase().contains(texto))
            .toList();
    }

    public Producto crear(ProductoRequest request) {
        validar(request);
        Long nuevoId = secuencia.incrementAndGet();
        Producto producto = new Producto(
            nuevoId,
            request.nombre(),
            request.precio(),
            request.stock()
        );
    }
}
```

```

        productos.put(nuevoId, producto);
        return producto;
    }

    public Producto actualizar(Long id, ProductoRequest request) {
        validar(request);
        Producto producto = buscarPorId(id);
        producto.setNombre(request.nombre());
        producto.setPrecio(request.precio());
        producto.setStock(request.stock());
        return producto;
    }

    public void eliminar(Long id) {
        Producto eliminado = productos.remove(id);
        if (eliminado == null) {
            throw new ProductoNoEncontradoException(id);
        }
    }

    private void validar(ProductoRequest request) {
        if (request.nombre() == null || request.nombre().isBlank()) {
            throw new IllegalArgumentException("El nombre del producto es obligatorio.");
        }
        if (request.precio() <= 0) {
            throw new IllegalArgumentException("El precio debe ser mayor que cero.");
        }
        if (request.stock() < 0) {
            throw new IllegalArgumentException("El stock no puede ser negativo.");
        }
    }
}

```

Aclaración importante

El almacenamiento en memoria se usa solo para esta práctica. En producción, los datos deben persistirse en una base de datos. El uso de `ConcurrentHashMap` y `AtomicLong` reduce problemas de concurrencia, pero no reemplaza una base de datos real.

7.6. Crear el controlador `ProductoController`

Crear el paquete controller y dentro el archivo `ProductoController.java`. Este controlador define los endpoints REST que serán consumidos por el cliente.

```

package com.utp.semana2.controller;

import java.net.URI;
import java.util.List;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import com.utp.semana2.dto.ProductoRequest;
import com.utp.semana2.model.Producto;
import com.utp.semana2.service.ProductoService;

@RestController
@RequestMapping("/api/productos")
public class ProductoController {

    private final ProductoService service;

    public ProductoController(ProductoService service) {
        this.service = service;
    }

    @GetMapping

```

```

public List<Producto> listar() {
    return service.listar();
}

@GetMapping("/{id}")
public Producto buscarPorId(@PathVariable Long id) {
    return service.buscarPorId(id);
}

@GetMapping("/buscar")
public List<Producto> buscarPorNombre(@RequestParam String nombre) {
    return service.buscarPorNombre(nombre);
}

@PostMapping
public ResponseEntity<Producto> crear(@RequestBody ProductoRequest request) {
    Producto creado = service.crear(request);
    URI location = URI.create("/api/productos/" + creado.getId());
    return ResponseEntity.created(location).body(creado);
}

@PutMapping("/{id}")
public Producto actualizar(@PathVariable Long id,
                           @RequestBody ProductoRequest request) {
    return service.actualizar(id, request);
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable Long id) {
    service.eliminar(id);
    return ResponseEntity.noContent().build();
}
}

```

7.7. Crear respuesta de error estándar

Cuando ocurre un error, el backend no debe devolver mensajes desordenados. Se creará un DTO simple para estandarizar la respuesta.

```

package com.utp.semana2.dto;

import java.time.LocalDateTime;

public record ErrorResponse(
    LocalDateTime fecha,
    int estado,
    String error,
    String mensaje,
    String ruta
) {
}

```

7.8. Crear manejador global de excepciones

Crear `GlobalExceptionHandler.java` dentro del paquete `exception`. Esta clase centraliza el manejo de errores para que el controlador se mantenga limpio.

```

package com.utp.semana2.exception;

import java.time.LocalDateTime;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import com.utp.semana2.dto.ErrorResponse;

import jakarta.servlet.http.HttpServletRequest;

@RestControllerAdvice
public class GlobalExceptionHandler {

```

```

    @ExceptionHandler(ProductoNoEncontradoException.class)
    public ResponseEntity<ErrorResponse> manejarNoEncontrado(
        ProductoNoEncontradoException ex,
        HttpServletRequest request) {

        ErrorResponse error = new ErrorResponse(
            LocalDateTime.now(),
            HttpStatus.NOT_FOUND.value(),
            "Not Found",
            ex.getMessage(),
            request.getRequestURI()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ErrorResponse> manejarSolicitudIncorrecta(
        IllegalArgumentException ex,
        HttpServletRequest request) {

        ErrorResponse error = new ErrorResponse(
            LocalDateTime.now(),
            HttpStatus.BAD_REQUEST.value(),
            "Bad Request",
            ex.getMessage(),
            request.getRequestURI()
        );
        return ResponseEntity.badRequest().body(error);
    }
}

```

8. Ejecutar la aplicación

Abrir la terminal integrada de VS Code y ejecutar:

```
mvn spring-boot:run
```

También se puede generar y ejecutar el JAR:

```
mvn clean package
java -jar target/semana2-0.0.1-SNAPSHOT.jar
```

Verificación esperada

La consola debe mostrar que Tomcat inició en el puerto 8080. Si aparece "Port 8080 already in use", cerrar el proceso anterior o cambiar server.port en application.properties.

9. Pruebas de la API

9.1. Prueba GET desde navegador

Abrir el navegador y colocar únicamente la URL, no escribir la palabra GET en la barra del navegador:

```
http://localhost:8080/api/productos
```

Resultado esperado: arreglo JSON con los productos iniciales.

9.2. Pruebas desde Postman o Thunder Client

Operación	Método	URL	Body JSON
Listar	GET	http://localhost:8080/api/productos	No aplica
Buscar por id	GET	http://localhost:8080/api/productos/1	No aplica
Buscar por nombre	GET	http://localhost:8080/api/productos/buscar?nombre=lap	No aplica
Crear	POST	http://localhost:8080/api/productos	{"nombre":"Monitor LG","precio":850,"stock":5}
Actualizar	PUT	http://localhost:8080/api/productos/1	{"nombre":"Laptop Lenovo Pro","precio":3900,"stock":7}

Eliminar	DELETE	http://localhost:8080/api/productos/2	No aplica
----------	--------	---------------------------------------	-----------

Error común

Si se prueba POST o PUT desde el navegador, no funcionará de forma cómoda porque el navegador ejecuta GET por defecto. Para enviar body JSON se debe usar Postman, Thunder Client o curl.

10. Explicación del flujo interno

Cuando se ejecuta una solicitud POST a /api/productos, ocurre el siguiente flujo:

1. El cliente envía JSON al endpoint.
2. ProductoController recibe el JSON mediante @RequestBody.
3. Spring convierte el JSON en un ProductoRequest usando Jackson.
4. El controlador llama a ProductoService.
5. ProductoService valida los datos y crea el producto.
6. El controlador devuelve respuesta HTTP 201 Created con el producto creado.

11. Validaciones y manejo de errores

Probar estos casos para observar respuestas controladas:

Caso	Solicitud	Resultado esperado
Producto inexistente	GET /api/productos/999	404 Not Found
Nombre vacío	POST con nombre vacío	400 Bad Request
Precio negativo	POST con precio -10	400 Bad Request
Stock negativo	POST con stock -1	400 Bad Request

12. Buenas prácticas aplicadas

- Usar rutas en plural para recursos REST: /api/productos.
- Separar controller y service.
- No colocar lógica de negocio directamente en el controller.
- Usar inyección por constructor y atributos final.
- Usar ResponseEntity cuando se necesita controlar el código HTTP.
- Validar los datos antes de crear o actualizar.
- Devolver errores consistentes mediante @RestControllerAdvice.
- No usar new ProductoService() dentro del controller.

13. Actividad guiada para el estudiante

Completar las siguientes mejoras:

1. Agregar endpoint GET /api/productos/stock-bajo?limite=10 para listar productos con stock menor o igual al límite.
2. Agregar endpoint GET /api/productos/precio-mayor?precio=100 para filtrar productos por precio.
3. Agregar validación para que el nombre tenga al menos 3 caracteres.
4. Modificar el mensaje de error cuando el producto no existe.
5. Agregar un producto inicial adicional en el constructor del servicio.

14. Preguntas de reflexión

1. ¿Qué diferencia existe entre @RestController y @Controller?

2. ¿Para qué sirve @RequestMapping?
3. ¿Cuál es la diferencia entre @PathVariable y @RequestParam?
4. ¿Por qué se recomienda inyección por constructor?
5. ¿Qué problema se evita al separar Controller y Service?
6. ¿Qué código HTTP se debe devolver al crear un recurso correctamente?
7. ¿Por qué POST y PUT requieren body JSON?
8. ¿Qué ocurre si Angular intenta consumir un endpoint que no existe?

15. Cierre conceptual

En esta práctica, el estudiante deja de ver Spring Boot como una clase principal que simplemente se ejecuta y empieza a comprenderlo como un framework que administra componentes, rutas, controladores y servicios. La idea central de la semana es que una API REST profesional debe organizarse por responsabilidades: el controller atiende HTTP, el service procesa la lógica y Spring se encarga de crear e inyectar las dependencias necesarias.

16. Referencias consultadas

- Spring Boot System Requirements. Documentación oficial de Spring Boot.
- Spring Initializr Reference Guide. Documentación oficial de Spring.
- Spring Framework Reference: Mapping Requests y REST Controllers.